

UTD Online Programming Contest
September 3rd 2016
Advanced Division

- Time Allowed: three hours
- **Each team must submit their attempted solutions from only one computer.** This is a PC2Team restriction.
- Answer the questions in any order.
- Use only Java, C or C++.
- Your program source code must be contained in one source file.
- Do not use the “package” construct in your Java code.
- Your programs must read from the data file given at the top of the problem description and output to System.out (cout for C/C++ programmers.)
- Make sure that your last line of output ends in a newline (i.e. is printed with System.out.println(...))
- **Do not copy solutions from the web.**
- Your solutions must be entirely yours, typed by you during the time allowed for the contest.
- As soon as you have solved a problem, submit **ONLY** the source file via your online PC2Team client.

Scoring

Equal points will be awarded to each question, even though they may not be equally difficult.

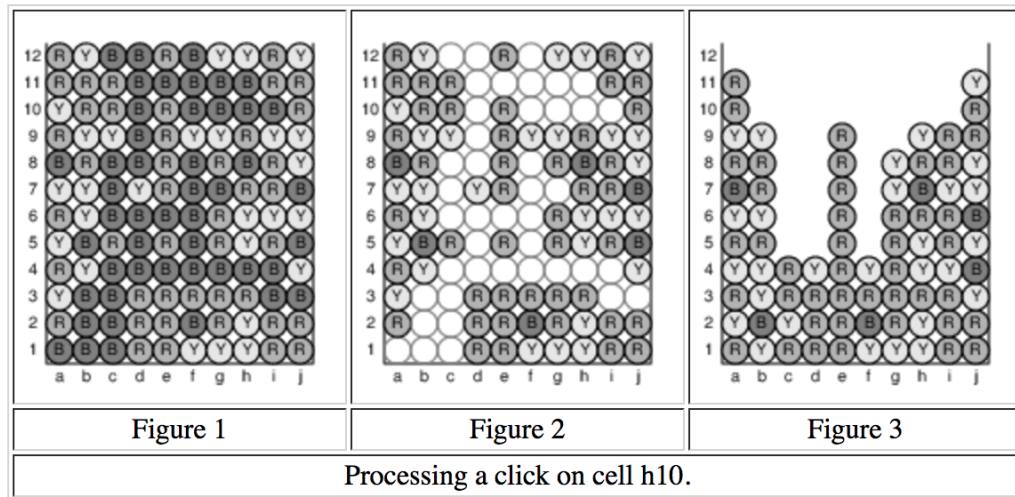
In order to break ties, we will use penalty points. The penalty points for a question will be zero if the question is not answered correctly. If a correct submission occurs for a question at time T minutes, then T penalty points will be added, plus 20 penalty points for each incorrect submission for that question.

A Cash Cow

Data File = A.txt

Time limit = 10 seconds

This single-player game has a board with 12 rows and 10 columns, as shown in Figure 1. We label the rows **1** through **12**, starting at the bottom, and the columns **a** through **j**, starting at the left. Each grid location can either have a colored circle or be empty. (We use uppercase characters to denote distinct colors, for example with **B=blue**, **R=red**, and **Y=yellow**.) On each turn, the player clicks on a circle. The computer determines the largest “cluster” to which that circle belongs, where a cluster is defined to include the initial circle, any of its immediate horizontal and vertical neighbors with matching color, those circles’ neighbors with matching colors, and so forth. For example, if a user were to click on the blue circle at cell (**h10**) in Figure 1, its cluster consists of those cells shown with empty circles in Figure 2.

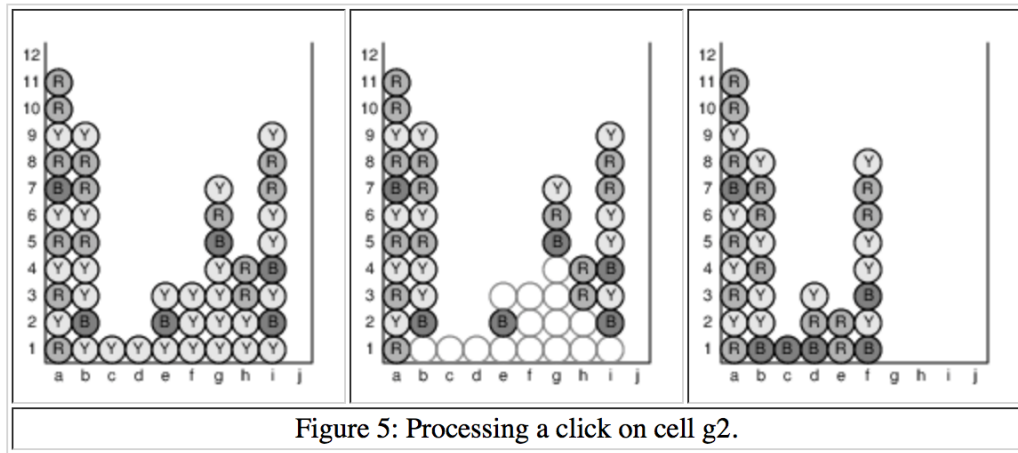
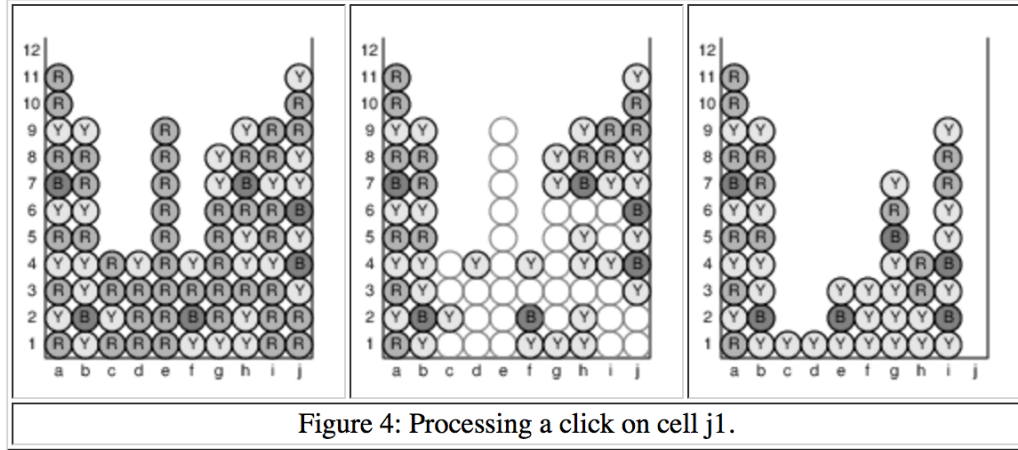


The player's turn is processed as follows. If the indicated grid cell belongs to a cluster of only one or two circles (or if there is no circle at that cell), the turn is wasted. Otherwise, with a cluster of 3 or more circles, all circles in the cluster are removed from the board. Remaining circles are then compacted as follows:

- 1 Circles fall vertically, to fill in any holes in their column.
- 2 If one or more columns have become empty, all remaining columns slide leftward (with each nonempty column remaining intact), such that they are packed against the left edge of the board.

For example, Figure 3 shows the board after the cluster of Figure 2 was removed after the click on (**h10**).

As another example, Figure 4 below, portrays the processing of a subsequent click on cell (**j1**). During that turn, column (**e**) becomes empty, and the resulting columns (**f**) through (**j**) slide to become columns (**e**) through (**i**), respectively. Figure 5 provides one further example in which several columns are compacted.



Input:

The input will consist of multiple games, each played with a new board. For each game, the input begins with a number T that denotes the number of turns that the player will be making, with $1 \leq T \leq 20$. Following that will be an initial board configuration, which always has 12 rows and 10 columns per row, with uppercase letters used to denote distinct colors. There will never be empty cells within the initial board. Following the presentation of the initial board will be T additional lines of input, each designating a cell of the grid; we rely on the coordinate system illustrated in the above figures, with a lowercase letter, from **a** to **j**, denoting a column and a number from **1** to **12** that denotes a row. We note that if a player clicks on a grid cell that does not currently have any circle, that turn is simply wasted.

The end of the entire input will be designated by a line with the number **0**.

Output:

For each game, output a single line designating the the number of circles that remain on the board after all of the player's turns are processed.

Sample Input	Sample Output
3	33
RYBBRBYRY	62
RRBBBBBR	2
YRRBRBBBR	
RYYBRYRY	
BRBBRBRBY	
YYBYRBBRRB	
RYBBBBRYYY	
YBRBRBRYRB	
RYBBBBBBY	
YBBRRRRBB	
RBBRRBRYRR	
BBRRYYYRR	
h 10	
j 1	
g 2	
3	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYBYYBBBBB	
YYBYYBBBBB	
c 2	
c 12	
g 1	
2	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYYYYBBBBB	
YYBYYBBBBB	
YYBYYBBBBB	
g 1	
c 12	4
0	

B DuLLs

Data File: B.txt
Time allowed = 10 seconds

In Windows, a DLL (or dynamic link library) is a file that contains a collection of pre-compiled functions that can be loaded into a program at runtime. The two primary benefits of DLLs are (1) only one copy of a DLL is needed in memory, regardless of how many different programs are using it at the same time, and (2) since they are separate from programs, DLLs can be upgraded independently, without having to recompile the programs that use them. (DLLs have their problems, too, but we'll ignore those for now.) Your job is to calculate the maximum memory usage when running a series of programs together with the DLLs they need.

The DLLs in our system are not very exciting. These dull DLLs (or DuLLs) each require a fixed amount of memory which never changes as long as the DuLL is in memory. Similarly, each program has its own fixed memory requirements which never change as long as the program is executing. Each program also requires certain DuLLs to be in memory the entire time the program is executing. Therefore, the only time the amount of memory required changes is when a new program is executed, or a currently running program exits. When a new program begins execution, all DuLLs required by that program that must be loaded into memory if they are not there already. When a currently running program exits, all DuLLs that are no longer needed by any currently running programs are removed from memory.

Remember, there will never be more than one copy of a specific DuLL in memory at any given time. However, it is possible for multiple instances of the same program to be running at the same time. In this case each instance of the program would require its own memory; however, the instances still share DuLLs in the same way two unrelated programs would.

Input:

The input consists of at least one data set, followed by a line containing only 0.

The first line of a data set contains three space separated integers $N P S$, where N is the number of DuLLs available, $1 \leq N \leq 20$, P is the number of programs which can be executed, $1 \leq P \leq 9$, and S is the number of state transitions recorded, $1 \leq S \leq 32$.

The next line contains exactly N space separated integers representing the sizes in bytes of each of the DuLLs, $1 \leq size \leq 1,000$. Each DuLL is implicitly labeled with a letter: 'A', 'B', 'C', , possibly extending to 'T'. Therefore the first integer is the size of 'A', the second integer is the size of 'B', and so on.

The next P lines contain information about each of the programs, one program per line. Each line contains a single integer representing the size of the program in bytes, $1 \leq size \leq 1,000$, followed by 1 to N characters representing the DuLLs required by that program. There will be a single space between the size of the program and the DuLL labels, but no spaces between the labels themselves. The order of the labels is insignificant and therefore undefined, but they will all be valid DuLL labels, and no label will occur more than once. Each program is implicitly labeled with an integer: 1, 2, 3, possibly extending to 9.

The final line of the data set will contain S space separated integers. Each integer will either be a positive number q , $1 \leq q \leq P$, indicating that a new execution of program q has begun,

or else it will be a negative number $-q$, $1 \leq q \leq P$, indicating that a single execution of program q has completed. The transitions are given in the order they occurred. Each is a valid program number; if it is a negative number $-q$ then there will always be at least one instance of program q running.

Output:

There is one line of output for each data set, containing only the maximum amount of memory required throughout the execution of the data set.

Sample Input	Sample Output
2 2 3 500 600 100 A 200 B 2 1 2 5 4 8 100 400 200 500 300 250 AC 360 ACE 120 AB 40 DE 2 3 4 -3 1 2 -2 1 0	1600 2110

C Black Vienna

Data File: C.txt
Time allowed = 10 seconds

This problem is based on the game of Black Vienna. In this version there are three players and 18 cards labeled A-R. Three of the cards are set aside (hidden) and form the “Black Vienna” gang. The remaining cards are shuffled and dealt to the players so that each player has 5 cards. Players never reveal their cards to each other. There is a separate deck of “interrogation cards” which contain three distinct letters in ascending order, like ACG or BHR. Turns rotate through players 1, 2, and 3. On each player/s turn, that player selects an interrogation card, puts it face up in front of another player, and that other player must indicate the total number of these cards being held, without saying which ones. All players see the result of the “interrogation”. The play continues until a player deduces the three cards in the “gang”.

For example, suppose the cards are distributed as follows, and the game then proceeds:

Player 1: DGJLP; Player 2: EFOQR; Player 3: ACHMN; Gang: BIK

Turn 1: Player 1 interrogates player 2 with BJK; answer 0
Turn 2: Player 2 interrogates player 3 with ABK; answer 1
Turn 3: Player 3 interrogates player 2 with DEF; answer 2
Turn 4: Player 1 interrogates player 2 with EIL; answer 1
Turn 5: Player 2 interrogates player 3 with FIP; answer 0
Turn 6: Player 3 interrogates player 1 with GMO; answer 1
Turn 7: Player 1 interrogates player 2 with OQR; answer 3
Turn 8: Player 2 interrogates player 3 with ADQ; answer 1
Turn 9: Player 3 interrogates player 1 with EGJ; answer 2

In fact, the game does not need to get to turn 9. With enough thought, player 1 can deduce after turn 8 that the gang is BIK. It is your job to analyze records of games and deduce the earliest time that the gang could be determined for sure.

Input:

The input will consist of one to twelve data sets, followed by a line containing only 0.

The first line of a dataset contains the number, t , of turns reported, $2 \leq t \leq 15$.

The next line contains four blank separated strings for the hands of players 1, 2, and 3, followed by the cards for the gang.

The remaining t lines of the data set contain the data for each turn in order. Each line contains three blank separated tokens: the number of the player interrogated, the string of interrogation letters, and the answer provided.

All letter strings will contain only capital letters from A to R, in strictly increasing alphabetical order. The same interrogation string may appear in more than one turn of a game.

Output:

There is one line of output for each data set. The line contains the single character ‘?’ if no

player can be sure of the gang after all the turns listed. If a player can determine the gang, the line contains the earliest turn after which one or more players can be sure of the answer.

Sample Input	Sample Output
9 DGJLP EFOQR ACHMN BIK 2 BJK 0 3 ABK 1 2 DEF 2 2 EIL 1 3 FIP 0 1 GMO 1 2 OQR 3 3 ADQ 1 1 EGJ 2 3 ABCDE FGHIJ KLMNO PQR 3 BKQ 1 1 ADE 3 2 CHJ 2 0	8 ?

D Towers

Data File: D.txt
Time allowed = 10 seconds

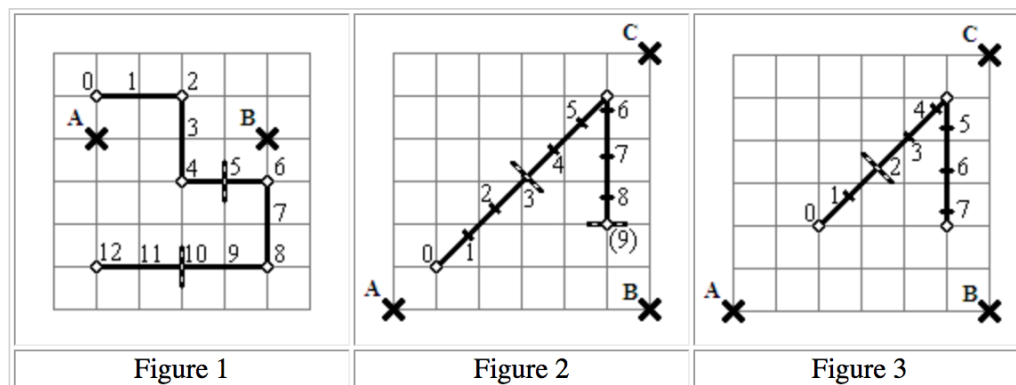
Cell phones generally provide service by connecting to a nearby cellular tower. At any given time there may be several towers in range. The cell phone, however, will only connect to the one with the best signal strength. The purpose of this problem is to track a traveler carrying a cell phone. At each mile marker along a road, note which tower the cell phone is using, and report when it is different from the previous marker.

The position of towers will be specified as $X - Y$ coordinates in miles relative to some arbitrary origin. The traveler will travel down a road composed of straight line segments laid end to end. The road will not intersect itself. Assume that there are markers occurring every mile along the road, with mile marker zero being at the starting point. If the road ends at least 0.5 miles past the last mile marker, the end of the road is labeled with the next mile. For instance if the road is 8.6 miles long, the endpoint is labeled as mile 9. If the road is 8.2 miles long, regular mile marker 8 is the last.

If d is the distance to a tower with power p , the signal strength for the tower will be calculated as p/d^2 , rounded to the nearest integer. A tower will never be placed at the position of a mile marker.

Consider the examples shown in the Figures below. Each shows a road and labeled mile markers and cell towers.

In Figure 1, where the segments of the road all follow the background grid, mile markers come at intersections in the grid. The cell towers A and B are at (1, 4) and (5, 4). Both have power 1,000. At mile markers 0 and 1 the strength of A is greater. At mile markers 2 - 4 the strength of A and B are equal. In such cases you are to note the cell tower with the label closer to the beginning of the alphabet, so it continues to be A. At mile markers 5 - 9 B is stronger. At mile 10 the towers are of equal strength, but the one with first letter is reported, A. Tower A remains strongest to the end. The long crossbars show the mile markers where the reported strongest tower is different than at the previous marker.



The example in Figure 2 has three towers: A at (0, 0) and C at (6, 6), both with power 1,000, while tower B at (6, 0) has a power of 600. The mile markers are at the tick marks. They show

the traveler's progression along the road. The mile markers are no longer at grid intersections because of the angled road. Initially tower A is the strongest. Tower C is strongest at mile markers 3 - 8. The end of the road at (5, 2) is more than a half mile from mile 8, so it is labeled as mile 9. At the endpoint, B is strongest, unlike at mile 8, so a report is made for the endpoint.

Figure 3 is similar to Figure 2, except it shifts the beginning of the road and changes the power of cell tower B to 300. Tower A starts as the strongest, and then at mile markers 2 - 7 tower C is strongest. The endpoint at (5, 2) is not considered, since it is less than .5 miles from mile marker 7, even though tower B is strongest at this endpoint.

Input:

The input consists of at least one data set, followed by a line containing only 0.

The first line of a data set contains two space separated integers T R , where

T is the number of towers, $1 \leq T \leq 10$

R is the number of line segments which comprise the road, $1 \leq R \leq 10$

The next T lines each contain three space separated integers representing the X-coordinate, Y-coordinate, and power of one tower, respectively. The towers are implicitly labeled 'A', 'B', 'C', and so on.

The next line contains $2(R + 1)$ integers which are the coordinates for the $R + 1$ points that define the road. The road starts on the first point and moves in straight line segments through all R remaining points.

All coordinates will be integers between 0 and 100, inclusive. No two coordinate pairs are equal. The power for a tower will be an integer between 1 and 1,000,000, inclusive.

Output:

There is one line of output for each road in the data set. The line consists of ordered pairs separated by a single space. The first element of a pair is a number representing a mile marker. The second element of the pair is a letter corresponding to the tower with the strongest signal. There are entries for mile 0 and every mile marker where the strongest tower recorded is different than at the previous mile marker. The ordered pairs are surrounded by parentheses, and the elements are separated by a comma, with no whitespace inside the ordered pair.

The sample input data corresponds to the Figures.

Sample Input	Sample Output
2 5	(0,A) (5,B) (10,A)
1 4 1000	(0,A) (3,C) (9,B)
5 4 1000	(0,A) (2,C)
1 5 3 5 3 3 5 3 5 1 1 1	
3 2	
0 0 1000	
6 0 600	
6 6 1000	
1 1 5 5 5 2	
3 2	
0 0 1000	
6 0 300	
6 6 1000	
2 2 5 5 5 2	
0	

E Got Change?

Data File: E.txt
Time allowed = 10 seconds

Given a denomination coin set, such as $\{1, 5, 10, 25, 50, 100\}$, what is the smallest number of coins needed to make up 87 cents? There is a greedy algorithm for this;

Use the largest denomination as many times as possible, then repeat the process with each lower denomination until the required amount is reached.

For 87c the minimum number of coins is five:

$$0 \times 100c + 1 \times 50c + 1 \times 25c + 1 \times 10c + 0 \times 5c + 2 \times 1c$$

But if the denomination set is $\{1, 5, 10, 12, 25, 50, 100\}$, the greedy method will not always give the best answer. Try 16c for example.

Given an amount to be changed and a denomination set, find the minimum number of coins to make up the given amount.

Input:

The input file begins with integer $T < 100$, on a line by itself, giving the number of test cases. Each test case follows on a single line and comprises space-separated integers. The amount, A , $A < 1000$, to be changed comes first, followed by the denomination set, given as a series of distinct integers in increasing order of size. The largest denomination will not be larger than 100.

Output:

For each test case print one line containing the minimum number of coins needed to make up the given amount.

Sample Input	Sample Output
3	
87 1 5 10 25 50 100	5
16 1 5 10 25 50	3
16 1 5 10 12 25	3